

Training the three arc.tex design proposals: what we ran and what came out

Run of 2026-09-29/30

1 What this is

`arc.tex` lists three design proposals. We trained and tested them on a local RTX 3060 Ti and on Kaggle T4 GPUs, including a few derivative models and parameter adjustments made after looking at medium-scale results. This file only reports what was run and what came out. Result files live under `output/arc_proposals/` on the PR branch. Scripts (all in `scripts/`):

- `run_structure_input_planning.py`, `run_structure_input_prediction.py`,
- `analyze_structure_route_selection.py`
- `run_actor_critic_litmus.py`, `run_action_reward_model.py`
- `run_latency_sweep2.py`, `make_result_plots.py` (figures and numbers in this file)

Mapping used between the proposals and the sections of `arc.tex`:

- **Structure-as-input world model** = `phys_feat` and `skip_feat0`: a known (possibly wrong) formula is given to the learned potential as an *input feature* (arc.tex §4).
- **Actor-critic and the oracle_act litmus** = replace REINFORCE by an actor-critic learner and ask whether the leaked positive control `oracle_act` now clearly beats `base` (arc.tex §2.5, §2.4, §2.8).
- **ChunkPPO with $\delta > 0$** = chunk-level PPO with a latency of δ ticks, arms `ignore`, `augment`, `wm`, `wm_passive`, `oracle` (arc.tex §3).

Summary.

Proposal		Outcome
Structure-as-input		Works on the pendulum (planning return -204 vs. -271 without the feature; ceiling -196), but generic unbounded features ($ q $, q^3 , q^4) do just as well, so the gain comes from an extrapolating input basis, not from the physics. A wrong formula used as a hard term is much worse.
Actor-critic <code>oracle_act</code>	+	The litmus passes: <code>oracle_act</code> beats <code>base</code> by $+1.0$ (12.8k episodes) and $+1.8$ (49.9k episodes); REINFORCE showed $+0.07$. A noop-future world model adds nothing beyond random features. A learned action-reward model gets part of the leaked gain at 12.8k episodes (about a quarter) and a small, not significant part at 49.9k.
ChunkPPO $\delta > 0$		No reproducible advantage of the forecast arms over <code>augment/ignore</code> . The one significant gap (intercept, $\delta = 2$) disappears with a second learner configuration; all gaps close at 10M ticks; on <code>pendulum_goal</code> the forecast arms are slightly worse.

2 Structure-as-input world model

2.1 What we did

Same network, data, optimiser and step count in every arm; only the scalar feature handed to the learned potential V changed. The learned model is a port-Hamiltonian network $\dot{x} = (J - R)\nabla H$ with $H = |p|^2/2 + \text{MLP}(q, \text{feature}(q))$ (feature route), or the feature added directly to H with a learned scale (*skip*, gate initialised at 0 or 1), or added as a fixed term (*hard*). The feature variants were: the proposal’s small-angle preset $q^2/2$; a rescaled $1.5q^2$; the true potential $-\cos q$ (an oracle, upper bound only); generic physics-free bases ($|q|$, $q^3/3$, $q^4/4$, the pair $[q^2/2, q^4/4]$); bounded oscillatory features ($\sin(5q + 0.7)$, $-\cos 2q$); a zero feature (same architecture, no information); and no feature at all (**hn**); plus a plain MLP.

Two testbeds:

- **Planning:** torque-limited pendulum swing-up ($|u| \leq 0.7$, start hanging, goal upright). Offline data: $N \in \{8, 32, 128\}$ short trajectories from initial angles $|\theta_0| < 1.5$ with random piecewise-constant torques, 2% observation noise, 2000 Adam steps, 10-step open-loop loss. The learned model is used inside a CEM model-predictive controller (3 iterations $\times$ 256 samples, horizon 60, replanning every step); the environment is stepped with the true dynamics. Score = return over 24 episodes; 6–12 seeds per cell.
- **Rollout prediction** on four tasks (Kepler with an unmodelled perturbation, Kepler with a wrong μ preset, falling body with quadratic drag, pendulum), $N = 64$ trajectories, 2% noise, 3000 steps, 5 seeds; metric = mean normalised RMSE over a 200–250-step open-loop rollout on held-out initial conditions inside the training range (ID) and outside it (OOD).

2.2 Results: planning

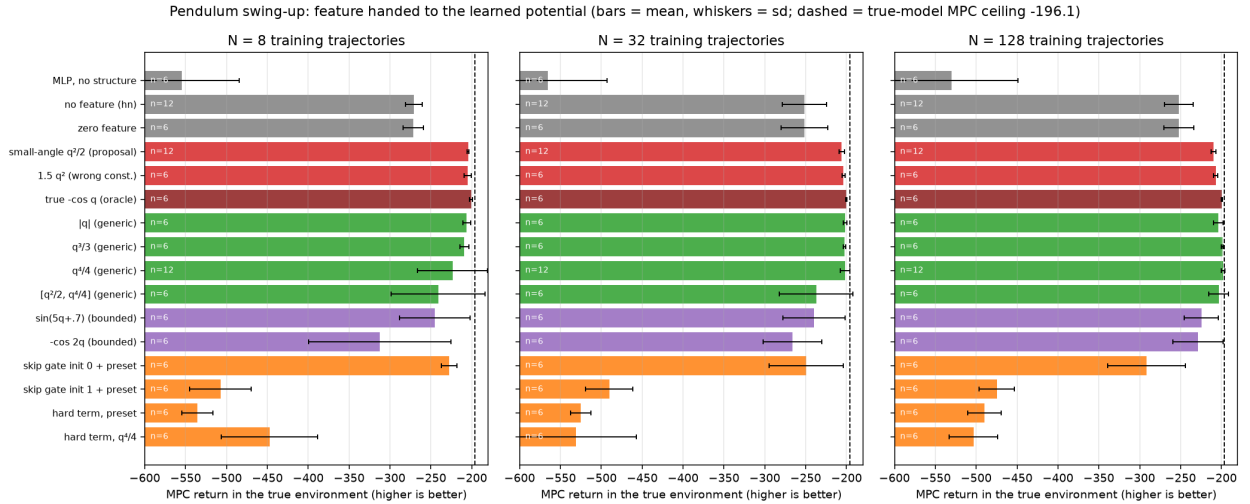


Figure 1: Pendulum swing-up MPC return in the true environment for each feature given to the learned potential, at three data sizes (mean $\pm$ sd over seeds; n printed in the bars; dashed line = true-model MPC ceiling -196.1). Source: `output/arc-proposals/structure_input/planning{,2}/`.

What we read from this:

Table 1: Selected planning results (mean $\pm$ sd; 12 seeds for *hn*, small-angle and $q^4/4$, 6 otherwise). Same source as the figure.

feature	$N = 8$	$N = 32$	$N = 128$
MLP (no structure)	-555 ± 70	-566 ± 73	-530 ± 82
none (<i>hn</i>)	-270.5 ± 10	-251.4 ± 27	-252.0 ± 18
zero feature	-271.3 ± 12	-251.5 ± 29	-252.0 ± 19
small-angle $q^2/2$ (proposal)	-204.3 ± 1.0	-206.0 ± 3.4	-209.4 ± 2.8
true $-\cos q$ (oracle)	-200.8 ± 1.6	-200.5 ± 0.8	-199.4 ± 0.8
$ q $ (generic)	-206.2 ± 4.8	-201.5 ± 2.3	-203.7 ± 5.8
$q^3/3$ (generic)	-209.0 ± 5.6	-202.4 ± 1.4	-198.7 ± 1.1
$q^4/4$ (generic)	-223 ± 43	-201.6 ± 5.9	-197.8 ± 2.2
$\sin(5q + 0.7)$ (bounded)	-245 ± 43	-240 ± 38	-225 ± 21
$-\cos 2q$ (bounded)	-312 ± 87	-266 ± 36	-229 ± 31
skip gate init 0 + preset	-227.5 ± 9	-249 ± 45	-292 ± 48
skip gate init 1 + preset	-507 ± 38	-490 ± 29	-475 ± 22
hard term, preset	-536 ± 19	-525 ± 13	-490 ± 20

- The proposal’s feature route gives about 65 return units over no feature at $N = 8$, and stays within 3–10 units of the true-model ceiling at every data size.
- The zero feature is identical to no feature, so the extra input dimension by itself does nothing.
- The same gain is obtained by $|q|$ and q^3 (no physical content) and by q^4 from $N = 32$ on. Bounded oscillatory features do not help. So the effect is an *unbounded, monotone* input that lets the network extrapolate to the upright region, which the data never visit; the physics preset is the most reliable member of that family at $N = 8$ (sd 1 vs. 43 for q^4).
- Using the same wrong formula as a fixed term or a skip connection starting open is far worse (–450 to –540); a skip starting closed is in between and gets worse with more data.

2.3 Results: rollout prediction

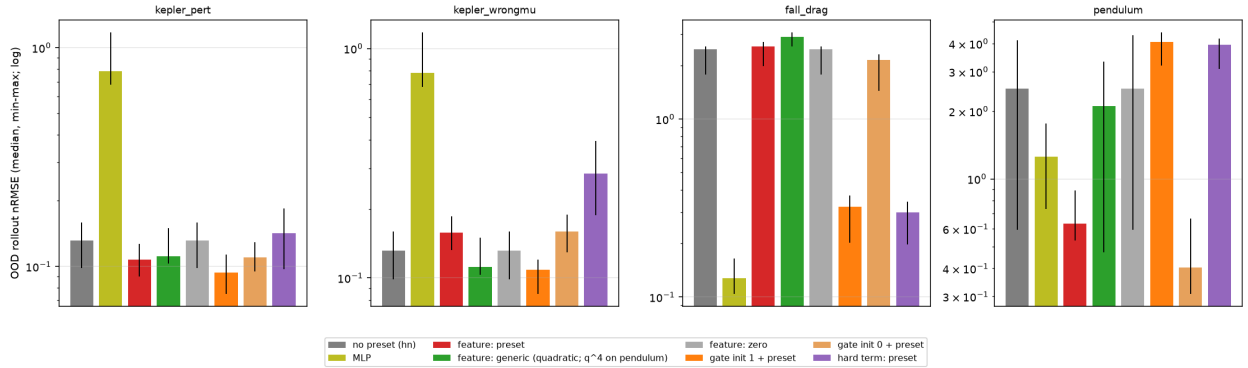


Figure 2: Out-of-distribution rollout error (median over 5 seeds, whiskers = min–max, log scale) per task and route. “Generic” is a quadratic on the Kepler and falling-body tasks and $q^4/4$ on the pendulum. Source: output/arc-proposals/structure_input/prediction{,2}/.

On Kepler the feature route is within noise of *hn* (which already has the Hamiltonian prior) and a generic quadratic does as well as the Newton preset. On the falling body the *hard* term wins,

Table 2: Median mean-nRMSE over 5 seeds, ID / OOD (lower is better).

route	Kepler pert.	Kepler wrong μ	falling + drag	pendulum
no preset (hn)	.140 / .132	.140 / .132	.467 / 2.47	.013 / 2.52
feature: preset	.149 / .107	.164 / .158	.672 / 2.56	.013 / .634
feature: generic	.139 / .111	–	1.28 / 2.90	.015 / 2.11
feature: zero	.140 / .132	.140 / .132	.467 / 2.47	.013 / 2.52
gate init 0 + preset	.150 / .110	.158 / .160	.512 / 2.15	.013 / .404
gate init 1 + preset (OOD only)	.093	.109	.321	4.08
hard term: preset	.187 / .142	.331 / .285	.149 / .298	.037 / 3.95

because the preset (gravity) is exactly right for the part it covers. On the pendulum the feature route (and the closed-gate skip) is what fixes extrapolation. No single route is best on all four tasks.

2.4 Derivative: choosing the route by held-out in-distribution error

For each task and seed we picked among {hn, feature, closed-gate skip, open-gate skip, hard term} using only the ID error, then read off the OOD error of the chosen route (no new training; reuse of the runs above).

Table 3: Median OOD nRMSE over seeds for each fixed route and for the ID-selected route.

task	hn	feature	skip0	skip1	hard	selected	hindsight best
falling + drag	2.473	2.564	2.148	0.321	0.298	0.321	0.298
Kepler pert.	0.132	0.107	0.110	0.093	0.142	0.093	0.093
Kepler wrong μ	0.132	0.158	0.160	0.109	0.285	0.110	0.109
pendulum	2.522	0.634	0.404	4.082	3.949	2.389	0.404
geometric mean	0.573	0.407	0.351	0.340	0.467	0.298	0.187

Selecting by ID error beats every fixed route on average, but on the pendulum all routes have ID error ≈ 0.013 , so the OOD lock-in of a wrong preset is invisible to it (it picks `hn` in 3 of 5 seeds; OOD 2.39 vs. 0.40 for the best route).

3 Actor-critic learner and the `oracle_act` litmus

3.1 What we did

Learner. New actor-critic code (`src/pipeline/candidate_actor_critic.py`); the REINFORCE code is untouched. PPO with clipped ratio (0.2), GAE($\lambda = 0.95$, $\gamma = 0.99$), entropy bonus 0.01, value coefficient 0.5, 4 epochs $\times$ 4 minibatches per update, 64 episodes of $T = 250$ per update, linear lr decay, AdamW. The critic is a small MLP (2×128 , tanh) on the 39-d observation summary plus t/T and is identical in every arm, so extra features never reach it. The actor is the same `transformer_branch` as in the REINFORCE study ($d = 256$, 277K parameters) on Craftax-Classic through `CraftaxFutureAdapter`. Budgets: 200 updates = 12,800 episodes (the REINFORCE budget diagnostic) and 780 updates = 49,920 episodes. Metric: mean return of 256 evaluation episodes of the final sampled policy; seeds are separate training runs. lr was 1e-3 unless stated.

Arms. **base**: 78 anticipation slots filled with zeros. **oracle_act**: slots hold the *true* one-step reward of each of the 17 actions (simulator branching at policy time; leaked positive control). **oracle**: true simulated noop future. **wm_full**: predictions of the trained passive world model from the full observation. **wm_untrain**: same network, random initialisation (control). Reference policies: a random policy and “argmax of the leaked one-step reward” (*greedy-1*), which scores mean return 3.17 (crafter score 2.76).

3.2 Litmus test

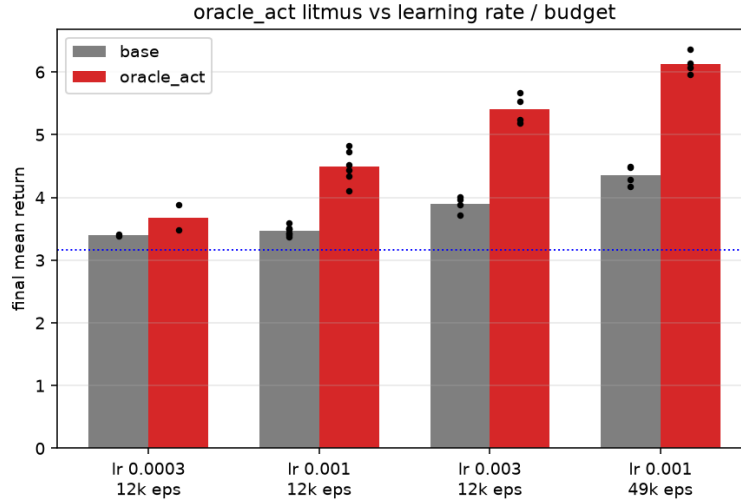


Figure 3: **base** vs. **oracle_act** for different learning rates and budgets (bars = mean, dots = seeds; dotted line = greedy-1 reference 3.17). Source: output/arc-proposals/ac.litmus-v*/ , ac.long*/.

Table 4: Final mean return, mean $\pm$ sd over seeds. REINFORCE numbers are the values quoted in arc.tex §2.5 (12,800 episodes, last-3-eval return).

learner, budget	base	oracle_act	gap	seeds
REINFORCE, 12.8k episodes	3.25 ± 0.14	3.32 ± 0.06	+0.07	4
actor-critic lr $3 \cdot 10^{-4}$, 12.8k	3.40 ± 0.03	3.68 ± 0.28	+0.28 (p=0.38)	2
actor-critic lr 10^{-3}, 12.8k	3.47 ± 0.07	4.49 ± 0.26	+1.03 (p= $1 \cdot 10^{-4}$)	8 / 6
actor-critic lr $3 \cdot 10^{-3}$, 12.8k	3.90 ± 0.13	5.41 ± 0.23	+1.51 (p= $1 \cdot 10^{-4}$)	4
actor-critic lr 10^{-3}, 49.9k	4.36 ± 0.16	6.13 ± 0.17	+1.78 (p= $5 \cdot 10^{-6}$)	4

At lr 10^{-3} every **oracle_act** seed is above every **base** seed (12.8k: min 4.10 vs. max 3.59; 49.9k: min 5.96 vs. max 4.50). Crafter score at 49.9k: 8.2 (**oracle_act**) vs. 4.8 (**base**). **oracle_act** is also well above the greedy-1 reference (3.17), so the learner uses the leaked signal beyond greedy use. The learner improves **base** itself with more episodes (3.47 $\rightarrow$ 4.36). Greedy-argmax evaluation is much lower than sampled evaluation for **base** (0.95 vs. 3.47 at 12.8k).

3.3 Does a passive (noop) world model help?

At 12.8k episodes the trained WM and even the true noop future are only +0.2 above **base**, and **wm_full**–**wm_untrain** = +0.07 (p=0.44), **oracle**–**wm_untrain** = +0.05 (p=0.58). At 49.9k episodes

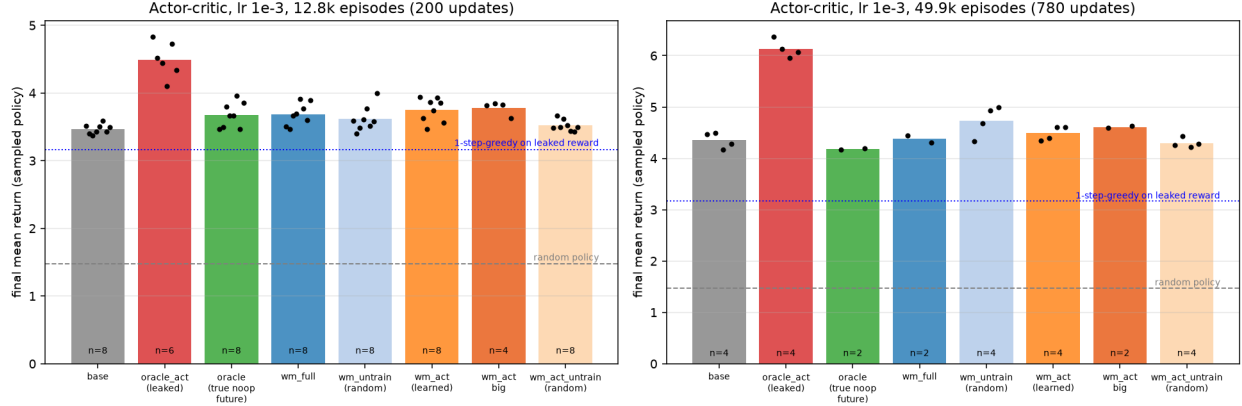


Figure 4: Final mean return of all actor-critic arms at $\text{lr } 10^{-3}$ (dots = seeds; dotted blue = greedy-1 reference; dashed grey = random policy). `wm_act` arms are the learned action-reward model described below. Source: `output/arc_proposals/ac_*/`.

Table 5: Noop-anticipation arms, $\text{lr } 10^{-3}$. Δ is relative to `base`; Welch p-values in brackets.

arm	12.8k episodes	Δ	49.9k episodes	Δ
<code>base</code>	3.47 ± 0.07 (n=8)	—	4.36 ± 0.16 (n=4)	—
<code>wm_untrain</code> (random)	3.62 ± 0.19 (8)	+0.15 (.06)	4.73 ± 0.30 (4)	+0.38 (.08)
<code>wm_full</code> (trained)	3.69 ± 0.16 (8)	+0.22 (.006)	4.38 ± 0.10 (2)	+0.02 (.86)
<code>oracle</code> (true noop future)	3.67 ± 0.19 (8)	+0.20 (.02)	4.18 ± 0.02 (2)	−0.17 (.11)
<code>oracle_act</code> (leaked)	4.49 ± 0.26 (6)	+1.03	6.13 ± 0.17 (4)	+1.78

the trained WM and the true noop future are no better than `base`, while the random-initialisation control is the highest of the three.

3.4 Derivative: a learned (deployable) version of the `oracle_act` signal

`oracle_act` needs the simulator at policy time. We trained an MLP (2×512 , GELU) that maps the raw 1345-d observation to the 17 one-step rewards, with labels from simulator branching used only offline (random-policy episodes, 80/20 split by episode), and served its predictions in the same 17 feature slots (arm `wm_act`; `wm_act_untrain` = same network with random weights).

Table 6: Held-out quality of the learned action-reward model. “Action-dependent” = the part of the reward that differs between actions in a state (about 18–21% of states).

	small (256 episodes)	big (1024 episodes)
train states / test states	27,187 / 7,951	116,978 / 28,716
R^2 over all 17 outputs	0.548	0.633
R^2 of the action-dependent part	0.487	0.620
best-action hit rate on action-dependent states	86.4%	87.3%
uniform random / always the globally best action	12.3% / 64.3%	14.9% / 61.8%

The learned signal is a real but small improvement: about a quarter of the leaked gain at 12.8k episodes (+0.28 of +1.03) and about 7–14% at 49.9k episodes (+0.13 to +0.25 of +1.78, not clearly

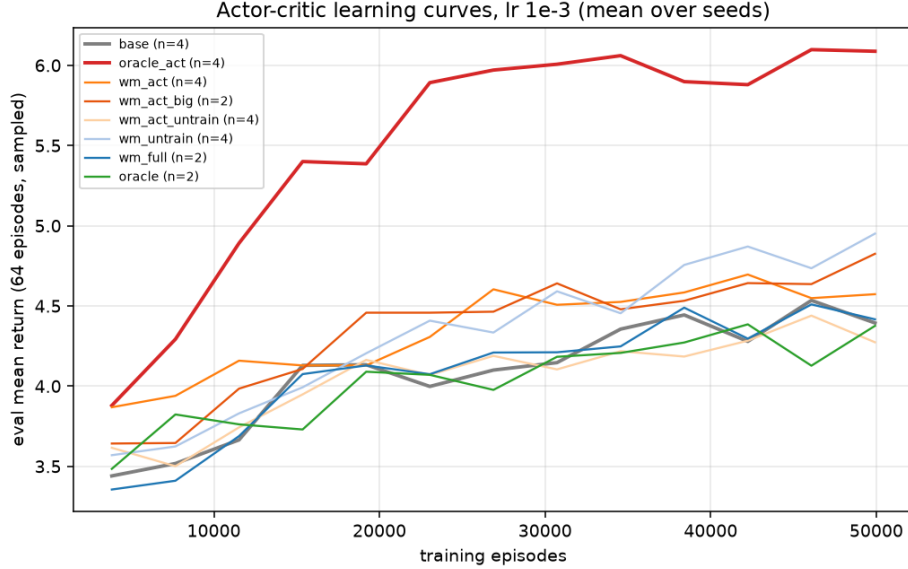


Figure 5: Learning curves of the 49.9k-episode runs (mean over seeds; evaluation of 64 episodes every 60 updates). Source: `output/arc_proposals/ac_stageb_long/, ac_long*/, ac_actreward*/`.

Table 7: Actor-critic with the learned action-reward features, lr 10^{-3} .

arm	12.8k episodes	49.9k episodes
base	3.47 ± 0.07 (n=8)	4.36 ± 0.16 (4)
wm_act_untrain (control)	3.52 ± 0.08 (8)	4.30 ± 0.09 (4)
wm_act (small model)	3.75 ± 0.18 (8); +0.28 vs. base (p=.002), +0.23 vs. control (p=.009)	4.49 ± 0.14 (4); +0.13 (p=.25)
wm_act (big model)	3.78 ± 0.10 (4); +0.31 (p=.003)	4.61 ± 0.03 (2); +0.25 (p=.04, n=2)
oracle_act (leaked)	4.49 ± 0.26 (6)	6.13 ± 0.17 (4)

significant). The larger model, despite better held-out R^2 , gives about the same policy result at 12.8k episodes.

4 ChunkPPO with $\delta > 0$

4.1 What we did

The existing chunk-level PPO (`src/pipeline/chunk_ppo.py`, unchanged) on two latency environments, driven by a new script that exposes every config field (`scripts/run_latency_sweep2.py`):

- **intercept**: unit arena, exogenous randomly turning target; catching gives +1 and ends the episode, 1 cost per step; 5 discrete moves; 200 ticks.
- **pendulum_goal**: torque-limited pendulum (5 torques) with a goal angle; 200 ticks.

Chunk length $k = 8$, latency $\delta \in \{0, 1, 2, 4, 8\}$ ticks (at $\delta = 0$ only **augment** is run; all arms see the same state there). Arms differ only in the actor’s input: **ignore** (snapshot only), **augment** (snapshot

+ committed prefix u_c), **wm** (factored world-model forecast of the state at $t_c + \delta$ from (o_c, u_c) , co-trained, with 12,800 passive reference-action ticks charged to its budget), **wm_passive** (forecast that ignores the committed actions; the original “noop model” proposal), **oracle** (exact future state including future noise; upper bound that no causal agent can reach). The critic, optimiser, budget and architecture are the same in all arms. Metric: **eval_return** = mean return of the first episode of each of 64 fresh environments under the stochastic policy (protocol v2).

Learner configurations. The pre-registered default (64 envs $\times$ 32 cycles $\times$ $k = 16,384$ ticks/update, lr $3 \cdot 10^{-4}$, 1M ticks $\Rightarrow$ about 61 PPO updates) learns **intercept** ($\delta = 0$ return 0.58) but does not learn **pendulum_goal** (return -178 at $\delta = 0$, about -180 for every arm and δ ; 1 seed). Pilots at $\delta = 0$, 2M ticks: config A (16 envs $\times$ 16 cycles = 2048 ticks/update, lr 10^{-3}) reached 0.70 on **intercept** and -63 on **pendulum_goal**; config B (32 envs $\times$ 16 cycles, lr $3 \cdot 10^{-4}$, entropy 0.003) reached 0.72 and -85 . Config A was used for the main sweeps; config B was then run on **intercept** as a robustness check.

4.2 Results: intercept

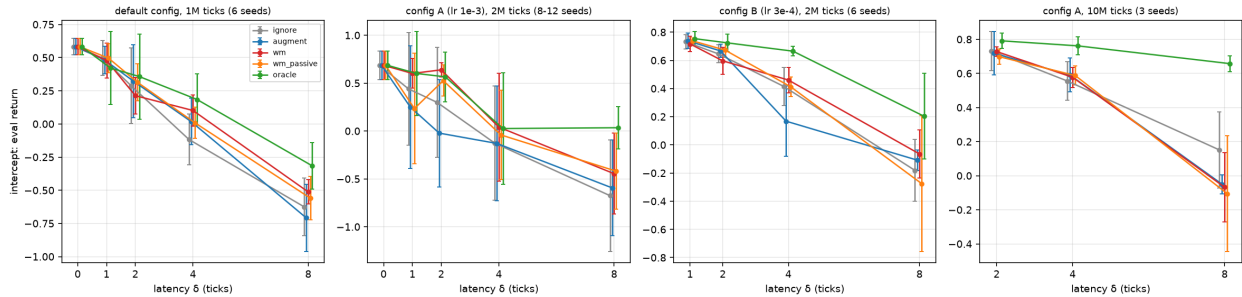


Figure 6: Intercept: evaluation return vs. latency δ for the five arms (mean $\pm$ sd over seeds). From left: default config at 1M ticks; config A at 2M ticks (8 seeds, 12 at $\delta = 2, 4$); config B at 2M ticks (6 seeds); config A at 10M ticks (3 seeds). Source: output/arc_proposals/latency/.

Permutation tests against **augment** (same setting and δ):

- Config A, $\delta = 2$ (12 seeds): **wm** +0.66 ($p < 0.001$; 12/12 seeds above 0.4 vs. 4/12 for **augment**), **wm_passive** +0.55 ($p=0.004$), **oracle** +0.59 ($p=0.004$). Config A, $\delta = 1, 4, 8$ (8/12/8 seeds): **wm** +0.35, +0.17, +0.15, none significant; **oracle** significant only at $\delta = 8$ (+0.63, $p=0.005$).
- Config B, $\delta = 2$: **wm** -0.07 ($p=0.14$), i.e. the gap is gone (**augment** 0.67 vs. 0.03 under config A). $\delta = 4$: **wm** +0.29 ($p=0.02$), but **ignore** +0.25 ($p=0.049$) and **wm_passive** +0.24 ($p=0.04$) show the same gain, and **wm**–**ignore** = +0.04. $\delta = 1, 8$: nothing significant except **oracle** at $\delta = 8$ (+0.31, $p=0.05$).
- 10M ticks: all causal arms are equal within noise at $\delta = 2$ (0.70–0.73) and $\delta = 4$ (0.56–0.59); **oracle** stays higher at $\delta = 4$ and 8 because it sees the target’s future random turns.
- **augment** vs. **ignore**: never significantly better in any setting; significantly worse only for config B at $\delta = 4$.

The cost of latency itself is clear: the best causal return falls from about 0.7–0.8 at $\delta = 0$ to about 0.7 at $\delta = 1$ –2, about 0.5–0.6 at $\delta = 4$ and about 0–0.15 at $\delta = 8$.

Table 8: Intercept, evaluation return (mean $\pm$ sd). n : A = 8 (12 at $\delta = 2, 4$), B = 6, 10M = 3, default = 6.

setting	δ	ignore	augment	wm	wm_passive	oracle
A, 2M ticks	0		.69 $\pm$.15			
	1	.44 $\pm$.59	.25 $\pm$.64	.60 $\pm$.15	.23 $\pm$.58	.60 $\pm$.44
	2	.30 $\pm$.58	-.02 $\pm$.56	.64 $\pm$.08	.53 $\pm$.16	.57 $\pm$.26
	4	-.13 $\pm$.60	-.13 $\pm$.60	.04 $\pm$.56	-.04 $\pm$.47	.03 $\pm$.58
	8	-.68 $\pm$.58	-.59 $\pm$.50	-.45 $\pm$.42	-.42 $\pm$.40	.03 $\pm$.22
B, 2M ticks	1	.73 $\pm$.05	.74 $\pm$.06	.72 $\pm$.06	.74 $\pm$.02	.75 $\pm$.05
	2	.65 $\pm$.06	.67 $\pm$.05	.60 $\pm$.09	.67 $\pm$.05	.72 $\pm$.06
	4	.42 $\pm$.13	.17 $\pm$.25	.46 $\pm$.09	.41 $\pm$.07	.67 $\pm$.03
	8	-.18 $\pm$.22	-.11 $\pm$.07	-.06 $\pm$.17	-.28 $\pm$.48	.20 $\pm$.30
A, 10M ticks	2	.73 $\pm$.12	.72 $\pm$.13	.73 $\pm$.03	.70 $\pm$.04	.79 $\pm$.04
	4	.56 $\pm$.11	.59 $\pm$.10	.58 $\pm$.06	.59 $\pm$.06	.76 $\pm$.05
	8	.15 $\pm$.22	-.05 $\pm$.06	-.07 $\pm$.20	-.11 $\pm$.34	.66 $\pm$.05
default, 1M ticks	0		.58 $\pm$.06			
	1	.50 $\pm$.13	.48 $\pm$.10	.48 $\pm$.13	.50 $\pm$.11	.42 $\pm$.28
	2	.29 $\pm$.29	.32 $\pm$.28	.21 $\pm$.14	.31 $\pm$.14	.36 $\pm$.32
	4	-.12 $\pm$.19	.02 $\pm$.17	.10 $\pm$.12	.01 $\pm$.12	.18 $\pm$.19
	8	-.63 $\pm$.22	-.71 $\pm$.25	-.51 $\pm$.09	-.56 $\pm$.16	-.32 $\pm$.18

4.3 Results: pendulum_goal

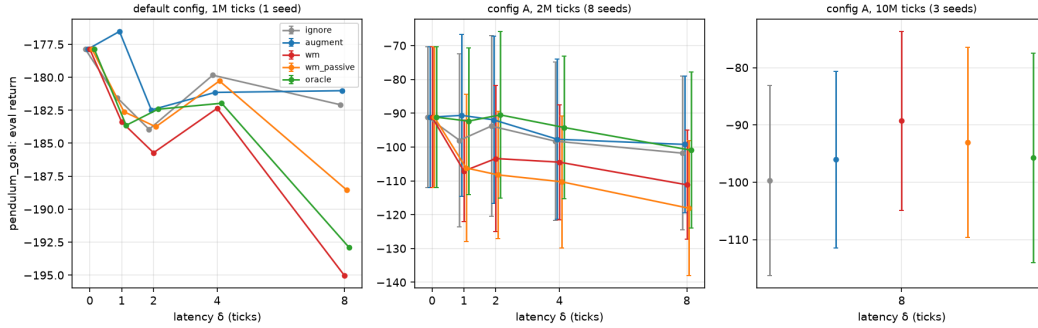


Figure 7: pendulum_goal: evaluation return vs. δ . Left: default config at 1M ticks (1 seed; no learning). Middle: config A at 2M ticks (8 seeds). Right: config A at 10M ticks, $\delta = 8$ only (3 seeds). Source: output/arc_proposals/latency/.

On pendulum_goal the forecast arms are -7 to -16 below **augment** at 2M ticks (none significant, p between 0.12 and 0.52 for **wm**) and **augment**, **ignore** and **oracle** are indistinguishable, i.e. latency up to $\delta = 8$ costs only a few return units here. At 10M ticks all arms are within one sd of each other.

5 Overall reading

- **Structure-as-input**: useful as an input feature where the network must extrapolate beyond its data, but on this testbed the benefit is provided by any unbounded monotone feature;

Table 9: pendulum_goal, config A, 2M ticks, 8 seeds (mean $\pm$ sd).

δ	ignore	augment	wm	wm_passive	oracle
0		-91.2 ± 20.8			
1	-98.0 ± 25.7	-90.6 ± 23.9	-107.1 ± 15.0	-106.2 ± 21.8	-92.4 ± 21.7
2	-93.8 ± 26.7	-92.0 ± 24.8	-103.4 ± 21.5	-108.2 ± 18.8	-90.5 ± 24.7
4	-98.3 ± 23.4	-97.7 ± 23.7	-104.5 ± 17.0	-110.3 ± 19.5	-94.2 ± 21.1
8	-101.8 ± 22.7	-99.2 ± 20.2	-111.2 ± 16.1	-118.1 ± 20.0	-101.0 ± 23.1
8 (10M ticks, 3 seeds)	-99.7 ± 16.6	-96.1 ± 15.4	-89.3 ± 15.6	-93.1 ± 16.6	-95.7 ± 18.3

hard-coding a wrong formula is harmful; a route chosen by ID error cannot detect OOD failure.

- **Actor-critic:** under PPO+GAE+critic ($\text{lr} \geq 10^{-3}$) the `oracle_act` litmus passes clearly, which the REINFORCE learner could not show. The noop-anticipation world model is not useful (even its oracle version); the learned action-reward model recovers part of the leaked gain at the smaller budget.
- **ChunkPPO** $\delta > 0$: the forecast arms show no gain that survives a change of learner configuration or a larger budget; the visible gaps come from `augment` failing to optimise at one particular δ .